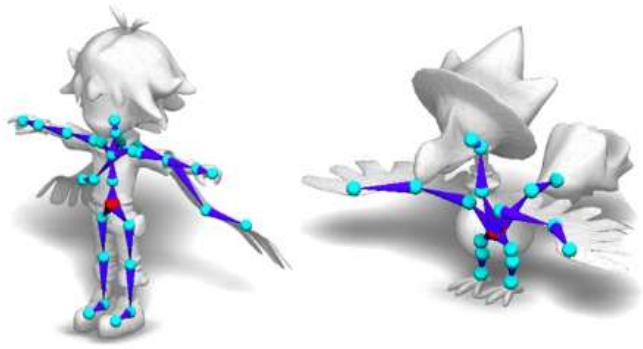
Hierarchical Character Animation

Kinematics, skinning, and motion

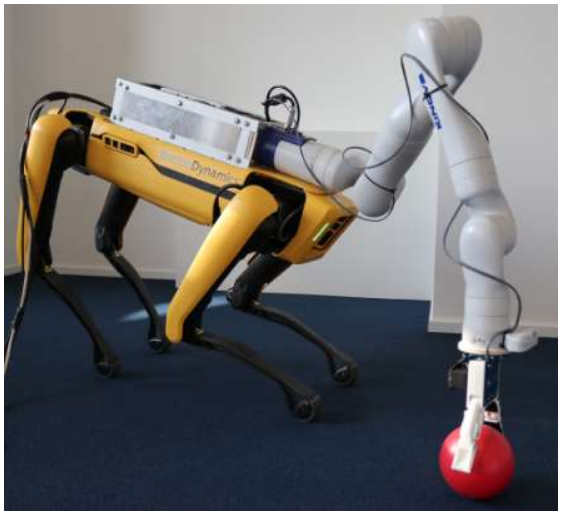
Please do not redistribute outside the classroom.

1

David Hahn, 2026-03-19



Xu et al., ACM TOG 39, 4, 58 (2020)



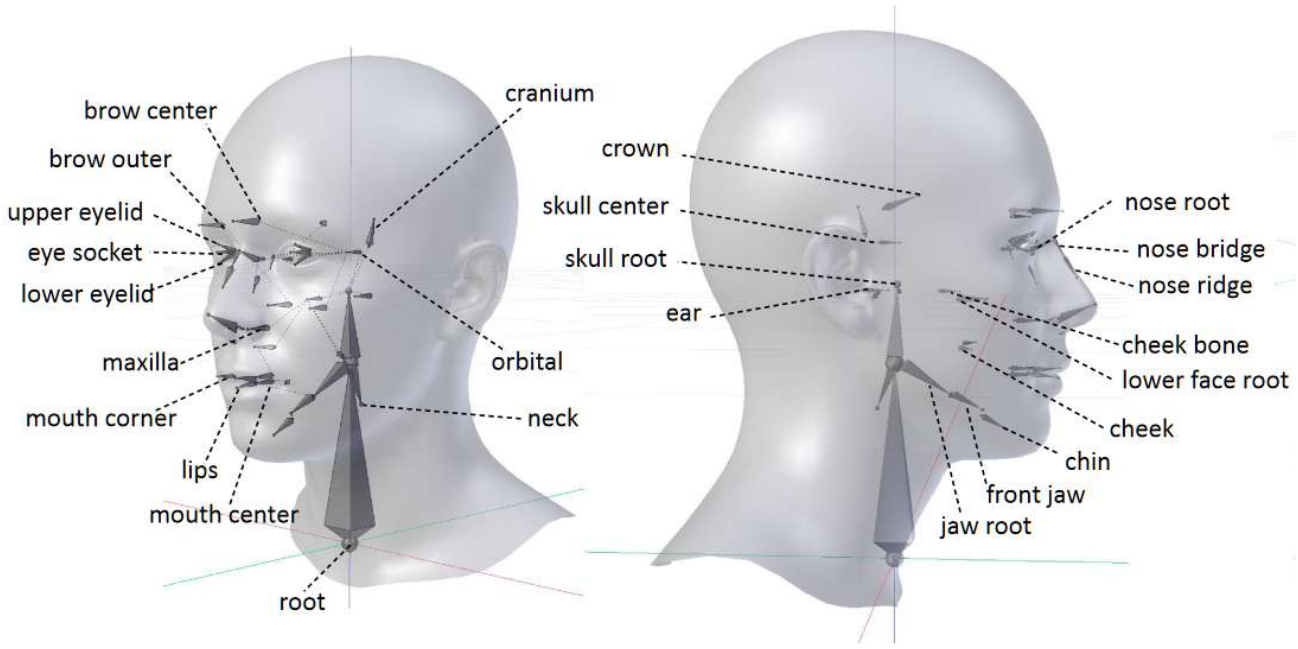
Zimmermann et al., IEEE ICRA 2021

2

Some motivational examples:

left: characters (meshes) with skeleton (kinematic rig), we want to use the skeleton to deform the mesh (skinning)

right: robotics application – we can represent the robot as an articulated body (same kinematic theory) to run simulations and plan motions etc.



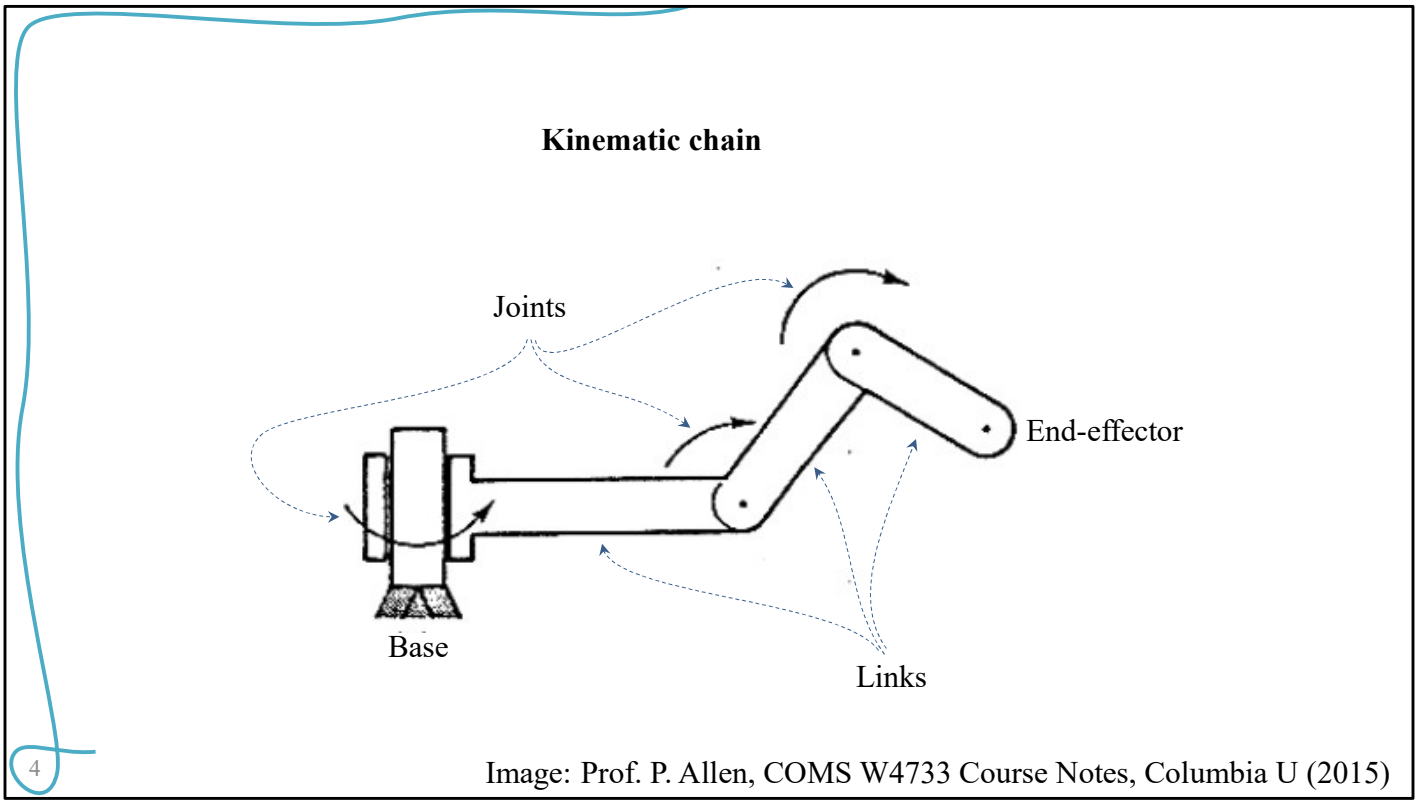
Vesdapunt et al., ECCV 2020 Lecture Notes in Computer Science

3

Another example

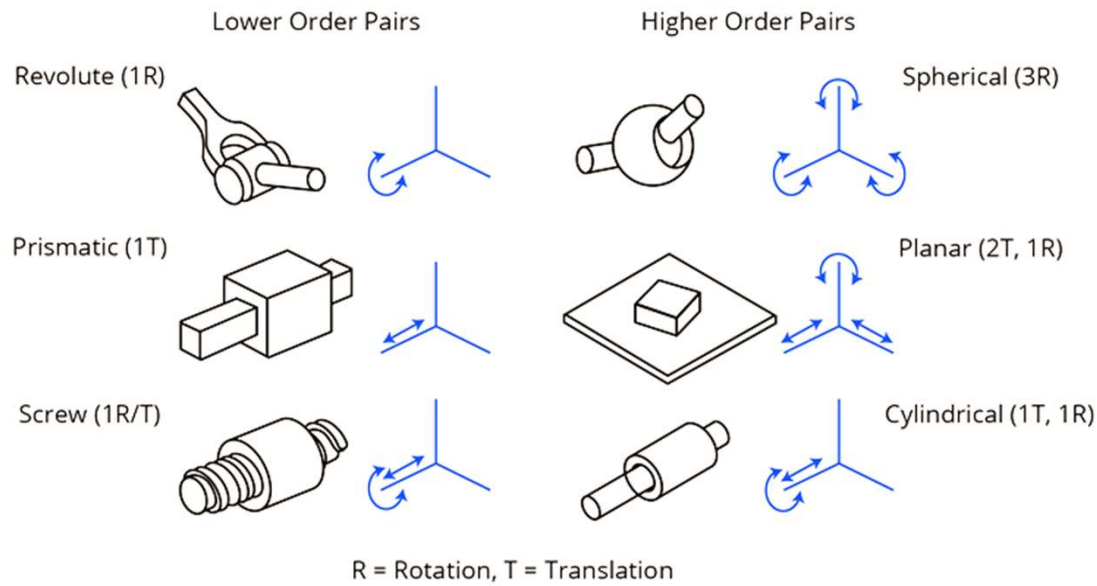
Note that the bones may not be drawn as connected segments, but they usually form a (logical) tree structure

We'll cover three main topics today:
(1) Kinematics of the skeleton itself (forward and inverse)
(2) Skinning deformation of the character mesh bound to the skeleton
(3) Animating these deformations over time (mostly using keyframes)



Definitions for the kinematic chain … links, joints, base, end-effector

Image: https://www.cs.columbia.edu/~allen/F15/NOTES/forwardspong.pdf



https://jychstar.blogspot.com/2017/09/robot-nd-a2-kinematics.html

5

Some joint types …

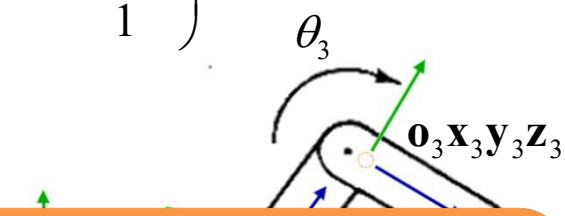
for now we don't worry too much about constraints

Local coordinate frame per link $\mathbf{o}_i\mathbf{x}_i\mathbf{y}_i\mathbf{z}_i$

Joint variables (degrees of freedom) per joint $\boldsymbol{\theta}_i$

Homogeneous transformation matrix $\mathbf{T}_i^{i-1} = \begin{pmatrix} \mathbf{R}_i(\boldsymbol{\theta}_i) & \mathbf{t}_i(\boldsymbol{\theta}_i) \\ 0 & 1 \end{pmatrix}$

$$\mathbf{T}_n^0 = \mathbf{T}_1^0\,\mathbf{T}_2^1\,\mathbf{T}_3^2 = \boxed{\prod_{i=1}^{n} \mathbf{T}_i^{i-1}}$$



$$\begin{pmatrix} \mathbf{R}_1 & \mathbf{t}_1 \\ 0 & 1 \end{pmatrix}\begin{pmatrix} \mathbf{R}_2 & \mathbf{t}_2 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_1\mathbf{R}_2 & \mathbf{R}_1\mathbf{t}_2 + \mathbf{t}_1 \\ 0 & 1 \end{pmatrix}$$

Forward kinematics [mostly] done.

6

Image:

math: forward kinematics -> joint space to world space transform matrices …

Understanding the product of homogeneous transformation matrices --> forward kinematics basically solved

$$\mathbf{o}_i\mathbf{x}_i\mathbf{y}_i\mathbf{z}_i = \begin{pmatrix} x_{i1} & y_{i1} & z_{i1} & o_{i1} \\ x_{i2} & y_{i2} & z_{i2} & o_{i2} \\ x_{i3} & y_{i3} & z_{i3} & o_{i3} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{T}_n^0 = \prod_{i=1}^{n} \mathbf{T}_i^{i-1}$$

$$\mathbf{o}_n\mathbf{x}_n\mathbf{y}_n\mathbf{z}_n = (\mathbf{o}_0\mathbf{x}_0\mathbf{y}_0\mathbf{z}_0)\,\mathbf{T}_n^0$$

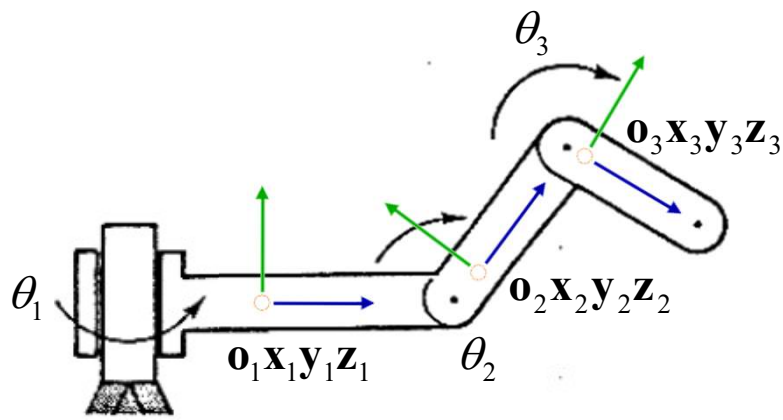


Image: Prof. P. Allen, COMS W4733 Course Notes, Columbia U (2015)

7

Define the base frame, put it all together – remember the sequence: left-to-right from base to endeffector

```
% base frame
oxyz_0 = eye(4);
oxyz_0(1,4)=0.2; oxyz_0(2,4)=0.1;

theta_1 = 30 /180*pi;
d_2     = 0.4;
theta_3 = 20 /180*pi;


% first joint -> rotate by theta_1 around x and const. offset of 0.3 along z
T_1 = [ axang2rotm([1 0 0  theta_1]), [0;0;0.3]; zeros(1,3), 1 ];

% second joint -> slide by d_2 along y
T_2 = [ eye(3), [0; d_2; 0] ; zeros(1,3), 1 ];

% third joint -> screw: rot. by theta_3 (z) and slide by 0.05*theta_3 (z)
T_3 = [ axang2rotm([0 0 1  theta_3]), [0;0; 0.05*theta_3]; zeros(1,3), 1 ];
```

```
oxyz_0 =

    1.0000         0         0    0.2
         0    1.0000         0    0.1
         0         0    1.0000
         0         0         0    1.0
```

8

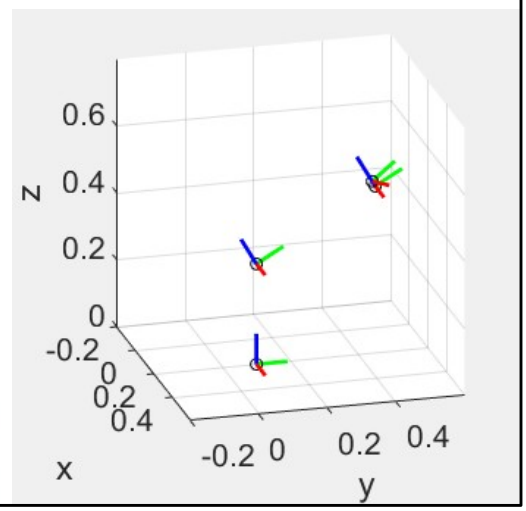
MatLab FK example

we use axang2rotm to get the rotation matrix given an axis and angle of rotation

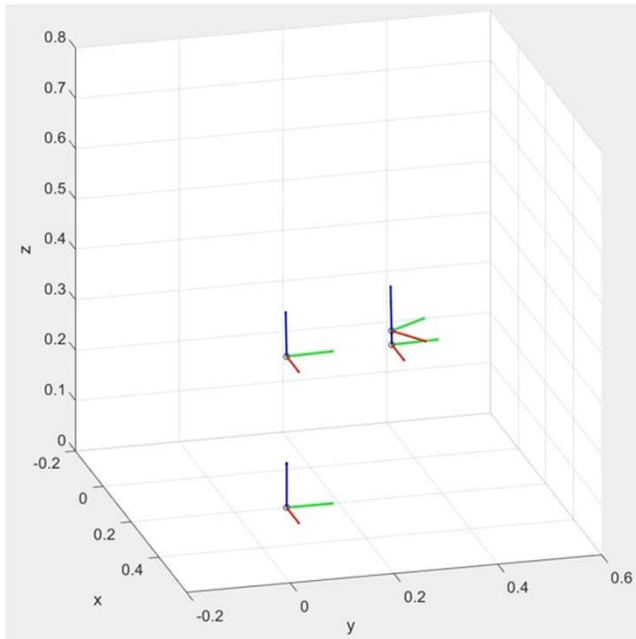
```
% first joint -> rotate by theta_1 around x and const. offset of 0.3 along z
T_1 = [ axang2rotm([1 0 0  theta_1]), [0;0;0.3]; zeros(1,3), 1 ];

% second joint -> slide by d_2 along y
T_2 = [ eye(3), [0; d_2; 0] ; zeros(1,3), 1 ];

% third joint -> screw: rot. by theta_3 (z) and slide by 0.05*theta_3 (z)
T_3 = [ axang2rotm([0 0 1  theta_3]), [0;0; 0.05*theta_3]; zeros(1,3), 1 ];
```

```
oxyz_1 = oxyz_0 * T_1;

oxyz_2 = oxyz_0 * T_1 * T_2; % == oxyz_1 * T_2

oxyz_3 = oxyz_2 * T_3; % == oxyz_0 * T_1 * T_2 * T_3
```

```
oxyz_3 =

    0.9397   -0.3420         0    0.2000
    0.2962    0.8138   -0.5000    0.4377
    0.1710    0.4698    0.8660    0.5151
         0         0         0    1.0000
```



9


0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
z
-0.2
0
0.2
0.4
x
-0.2
0
0.2
0.4
0.6
y

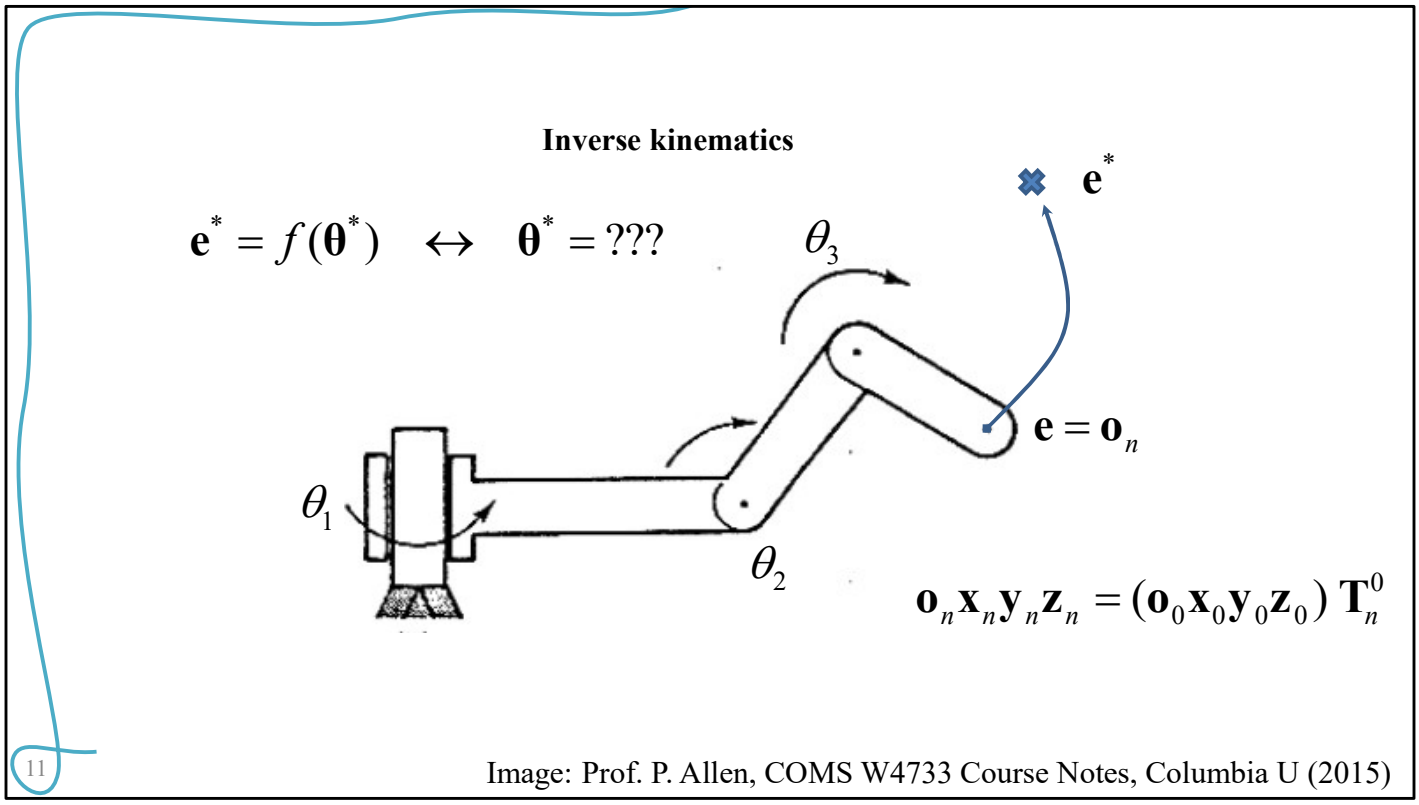


Image: Prof. P. Allen, COMS W4733 Course Notes, Columbia U (2015)

Given a target position of the endeffector, how can we find the joint angles that actually move it there (or as close as we can manage at least)?

For details on inverse kinematics, see: https://www.cs.cmu.edu/~15464-s13/lectures/lecture6/IK.pdf
STAR (2017): http://andreasaristidou.com/publications/papers/IK_survey.pdf
and also:
https://homes.cs.washington.edu/~todorov/courses/cseP590/06_JacobianMethods.pdf

$$\mathbf{e} = f(\boldsymbol{\theta}): \quad \text{find } \boldsymbol{\theta}^* \text{ s.t. } f(\boldsymbol{\theta}^*) \text{ is as close as possible to } \mathbf{e}^*$$

"Jacobian transpose" method: gradient descent for $\min_{\boldsymbol{\theta}} \Phi(\boldsymbol{\theta}), \quad \Phi = 0.5\left\|\mathbf{e}^* - f(\boldsymbol{\theta})\right\|^2$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \Delta\boldsymbol{\theta}$$

$$\Delta\boldsymbol{\theta} = -\alpha \frac{d\Phi}{d\boldsymbol{\theta}}^{\mathsf{T}} = \alpha \mathbf{J}^{\mathsf{T}}(\mathbf{e}^* - f(\boldsymbol{\theta}_k))$$

$$\mathbf{J} = \left.\frac{df}{d\boldsymbol{\theta}}\right|_{\boldsymbol{\theta}_k}$$

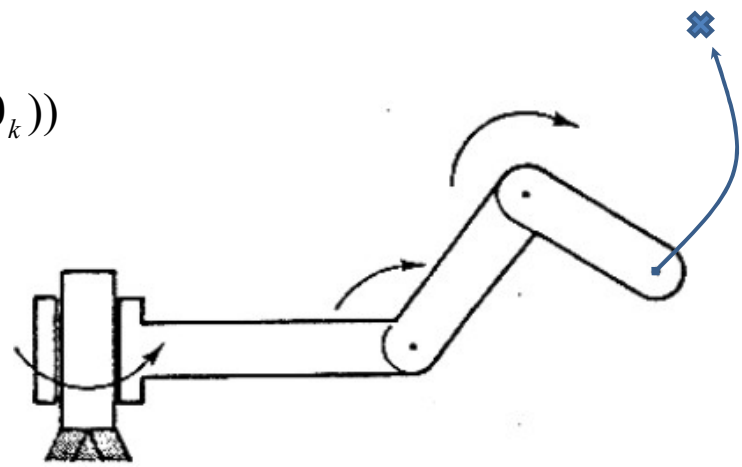


Image: Prof. P. Allen, COMS W4733 Course Notes, Columbia U (2015)

12

Formulating the question as an optimization problem and using simple gradient descent we arrive at the Jacobian transpose method.

This is a bit slow to converge, but easy to implement (we need to compute the Jacobian, but not much else).

Based on the same gradient, other first-order optimization methods could be used (Gauss-Newton, Powell's dog leg, L-BFGS, etc.)

Also, optimization algorithms are well suited to incorporating various constraints (i.e. joint limits).

$$\mathbf{e} = f(\boldsymbol{\theta}): \quad \text{find } \boldsymbol{\theta}^* \text{ s.t. } f(\boldsymbol{\theta}^*) \text{ is as close as possible to } \mathbf{e}^*$$

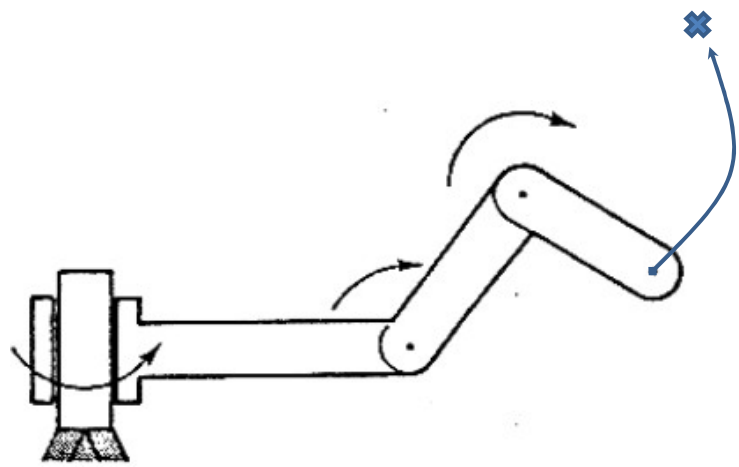
"Jacobian pseudo-inverse" method: Gauss-Newton $\min_{\boldsymbol{\theta}} \Phi(\boldsymbol{\theta}), \quad \Phi = 0.5\left\|\mathbf{e}^* - f(\boldsymbol{\theta})\right\|^2$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \Delta\boldsymbol{\theta}$$

$$\mathbf{J}^\mathsf{T}\mathbf{J}\,\Delta\boldsymbol{\theta} = \mathbf{J}^\mathsf{T}(\mathbf{e}^* - f(\boldsymbol{\theta}_k))$$

$$\Delta\boldsymbol{\theta} = \mathbf{J}^+(\mathbf{e}^* - f(\boldsymbol{\theta}_i))$$

$$\mathbf{J}^+ = \lim_{\delta\to 0}(\mathbf{J}^\mathsf{T}\mathbf{J} + \delta\mathbf{I})^{-1}\mathbf{J}^\mathsf{T}$$



13

Image: Prof. P. Allen, COMS W4733 Course Notes, Columbia U (2015)

Applying Gauss-Newton optimization instead yields the "Jacobian pseudo-inverse" method.

J^+ is known as the Moore-Penrose pseudo-inverse.

In practice, we could solve the linear system for the Gauss-Newton step (with a solver that is sufficiently stable),

or use a non-vanishing damping constant δ instead of the analytical pseudo-inverse (-> Levenberg-Marquardt optimizer).

```
% first joint -> rotate by theta_1 around x and const. offset of 0.3 along z
T_1 = [ axang2rotm([1 0 0  theta_1]), [0;0;0.3]; zeros(1,3), 1 ];

% second joint -> slide by d_2 along y
T_2 = [ eye(3), [0; d_2; 0] ; zeros(1,3), 1 ];

% third joint -> screw: rot. by theta_3 (z) and slide by 0.05*theta_3 (z)
T_3 = [ axang2rotm([0 0 1  theta_3]), [0;0; 0.05*theta_3]; zeros(1,3), 1 ];
```

$$\mathbf{e} = f(\boldsymbol{\theta}) \quad \ldots \quad \mathbf{J} = df / d\boldsymbol{\theta}? \quad \boldsymbol{\theta} = (\theta_1, d_2, \theta_3)^{\mathsf{T}}$$

```
e = eye(3,4)* oxyz_0 * T_1 * T_2 * T_3 *[0;0;0;1];
```

Easy to differentiate

T_1 =

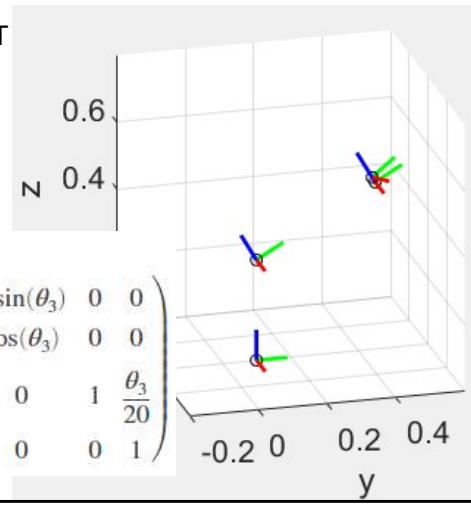
$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta_1) & -\sin(\theta_1) & 0 \\ 0 & \sin(\theta_1) & \cos(\theta_1) & \frac{3}{10} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

T_2 =

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & d_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

T_3 =

$$\begin{pmatrix} \cos(\theta_3) & -\sin(\theta_3) & 0 & 0 \\ \sin(\theta_3) & \cos(\theta_3) & 0 & 0 \\ 0 & 0 & 1 & \frac{\theta_3}{20} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



14

How to compute the Jacobian?

Note: so far we considered θ to be degrees of freedom associated with a specific joint.
In more complicated character rigs, often a combination of multiple joint DOFs is (partially) controlled through a single user input (called “animation variable”, or “avar”).

In principle IK, or even physics-based models can be formulated and approximately solved through avars as well, see e.g. “Rig-space physics”, SigGraph 2012.

… or just use symbolic math tools

```
syms theta_1 d_2 theta_3 real;

% R_1 = axang2rotm([1 0 0  theta_1]) <--- not for symbolic input
k   = [1 0 0];   a = theta_1;
K_  = [0.0, -k(3), k(2); k(3), 0.0, -k(1); -k(2), k(1), 0.0];
R_1 = eye(3) + sin(a)*K_ + (1.0-cos(a))*K_*K_;

% same for R_3 = axang2rotm([0 0 1  theta_3])
k   = [0 0 1];   a = theta_3;
K_  = [0.0, -k(3), k(2); k(3), 0.0, -k(1); -k(2), k(1), 0.0];
R_3 = eye(3) + sin(a)*K_ + (1.0-cos(a))*K_*K_;

T_1 = [ R_1, [0;0;0.3]; zeros(1,3), 1 ];
T_2 = [ eye(3), [0; d_2; 0] ; zeros(1,3), 1 ];
T_3 = [ R_3, [0;0; 0.05*theta_3]; zeros(1,3), 1 ];
```

```
% base frame
oxyz_0 = eye(4); oxyz_0(1,4)=0.2; oxyz_0(2,4)=0.1;

% end-effector position
e = eye(3,4) * oxyz_0 * T_1 * T_2 * T_3 * [0;0;0;1];

J = jacobian(e, [theta_1; d_2; theta_3])
```

J =

$$\begin{pmatrix} 0 & 0 & 0 \\ -d_2\sin(\theta_1) - \frac{\theta_3\cos(\theta_1)}{20} & \cos(\theta_1) & -\frac{\sin(\theta_1)}{20} \\ d_2\cos(\theta_1) - \frac{\theta_3\sin(\theta_1)}{20} & \sin(\theta_1) & \frac{\cos(\theta_1)}{20} \end{pmatrix}$$

15

How to compute the Jacobian? …. Use symbolic math tools, they make life easier.

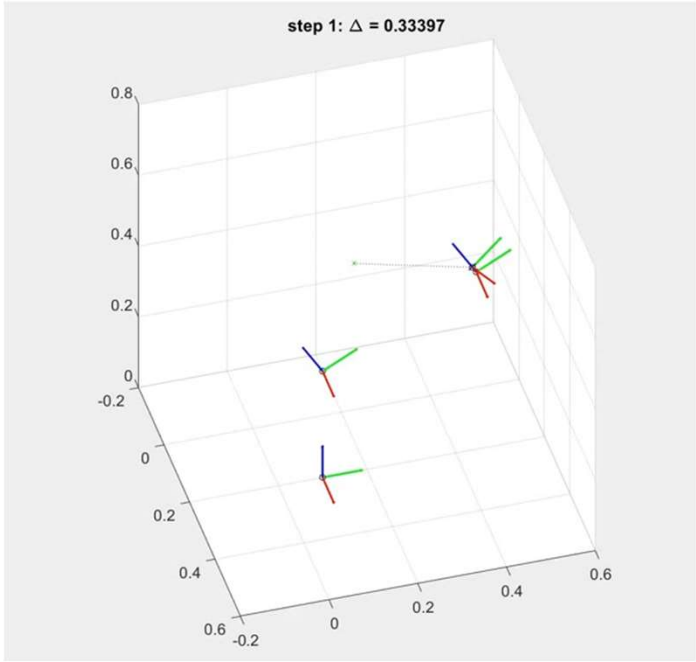
The resulting expression can be exported to code.

Nowadays, many libraries also provide automatic differentiation at runtime or compile-time (JAX, pytorch, slang.d, …)

```
e_target = [0.1 ; 0.2 ; 0.5];

% from symbolic math tool ...
J_= @(theta_1,d_2,theta_3) reshape([0.0,-d_2.*sin(theta_1)-theta_3.*cos(theta_1).*(

for step = 1:5
    T_1 = [ axang2rotm([1 0 0  theta_1]), [0;0;0.3]; zeros(1,3), 1 ];
    T_2 = [ eye(3), [0; d_2; 0] ; zeros(1,3), 1 ];
    T_3 = [ axang2rotm([0 0 1  theta_3]), [0;0; 0.05*theta_3]; zeros(1,3), 1 ];

    e = eye(3,4)* oxyz_0 * T_1 * T_2 * T_3 *[0;0;0;1]; % end-effector position

    J = J_(theta_1,d_2,theta_3);
    JtJ = J'*J;
    dt = (JtJ + 1e-3*eye(3)) \ (J'*( e_target - e ));

    theta_1 = theta_1 + dt(1);
    d_2 = d_2 + dt(2);
    theta_3 = theta_3 + dt(3);
end
```

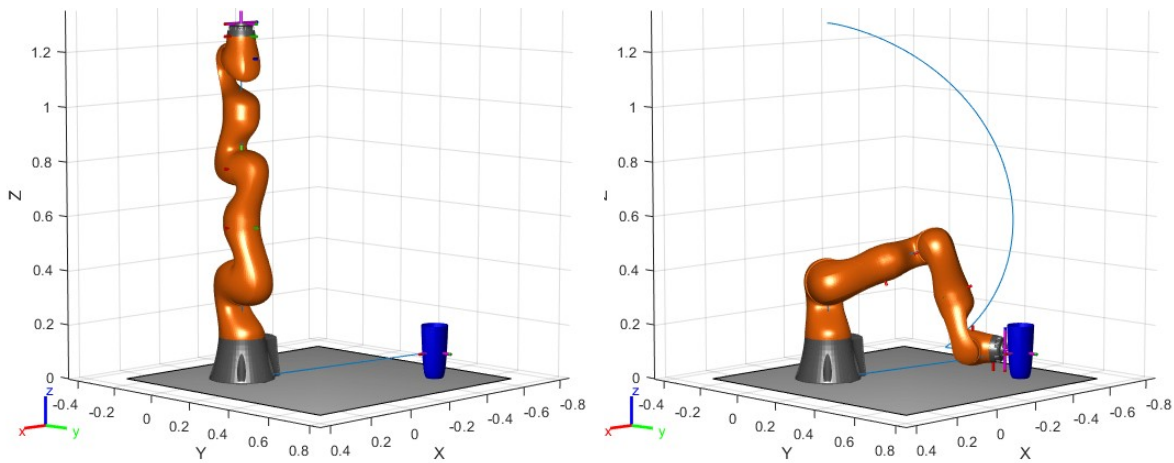
16

A simple example …



17

A simple example …

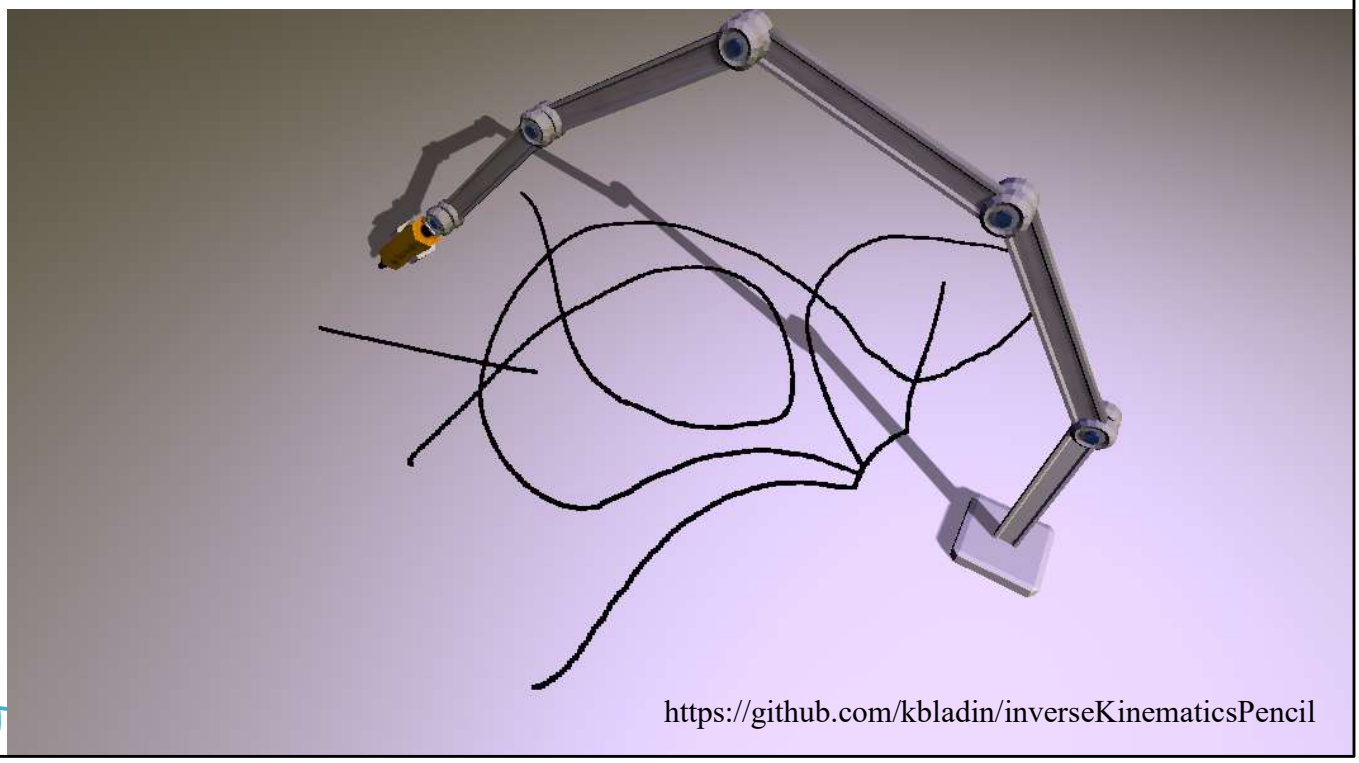


18 https://www.mathworks.com/help/robotics/ug/plan-a-reaching-trajectory-with-kinematic-constraints.html

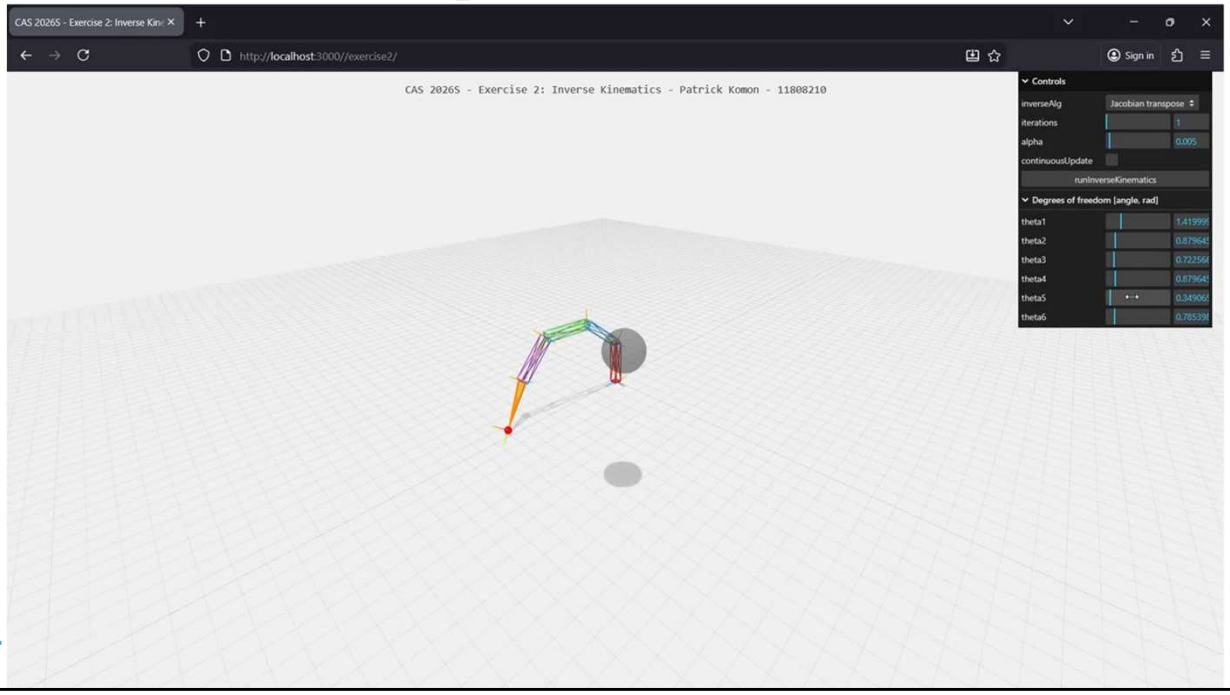
MatLab has an IK solver as part of its Robotics System Toolbox for experimentation

It also supports constraints on joint angles

With rigid parts (→ articulated body) drawing an separate mesh over each link is straightforward. Later we'll look into deforming continuous meshes accordingly.


https://github.com/kbladin/inverseKinematicsPencil

Exercise: FK & IK implementation



In the exercise assignment you'll implement forward and inverse kinematics in a JS framework prepared by our tutors. In the end the solution should look more or less like shown in this video.

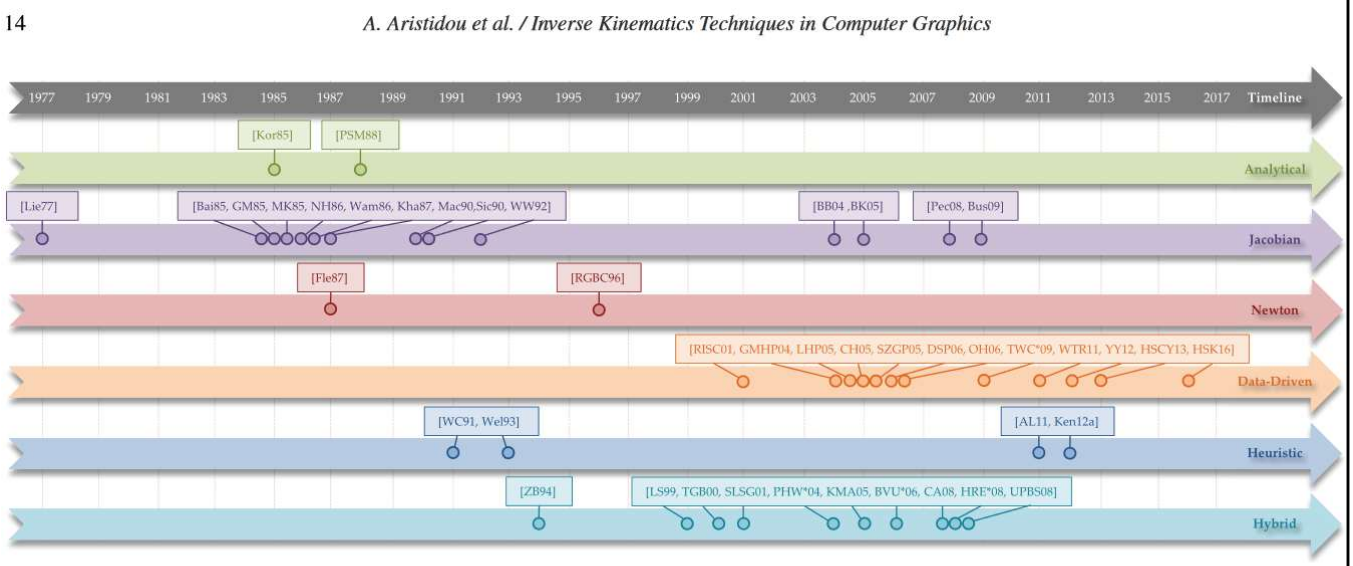


Figure 4: *A timeline showing how the IK approaches have progressed over the years by examining publication years of key methods.*

Aristidou et al., CGF (2017)

21

There are lots of other algorithms to solve IK problems.

In graphics (like games, or interactive animation tools) fast heuristics (like cyclic coordinate descent, or FABRIK) are sometimes more convenient to work with.

More references:

https://dl.acm.org/doi/10.1109/HUMANOIDS.2015.7363472

http://www.andreasaristidou.com/publications/papers/FABRIK.pdf

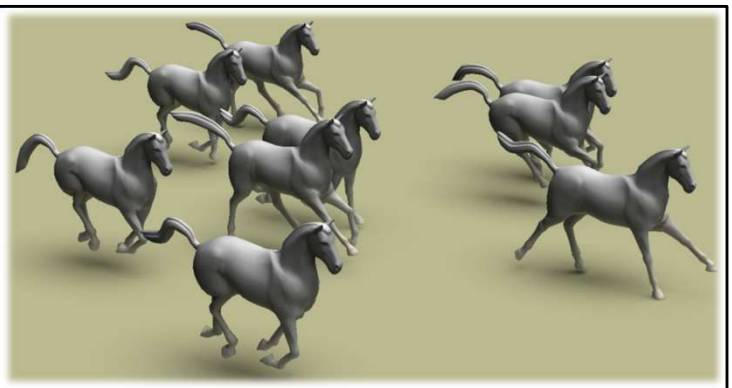
http://andreasaristidou.com/publications/papers/IK_survey.pdf

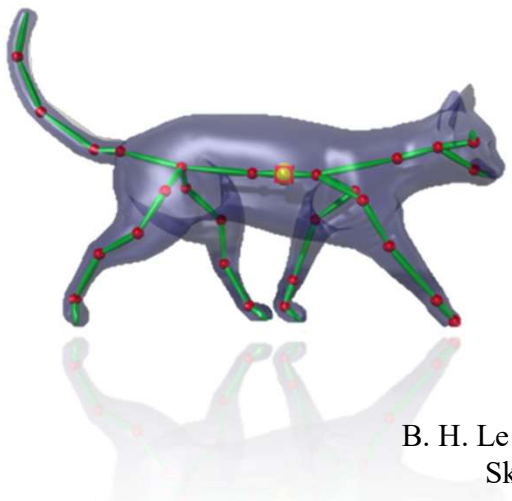
Done:
Kinematic chain (links, joints)
Forward and inverse kinematics
Character skeleton → collection of kinematic chains

Next:
Skinning → deforming a mesh to “follow” the skeleton
Animation via key frames, motion capture basics

end of part 1

James & Twigg, Skinning Mesh Animations
(SIGGRAPH 2005)
http://graphics.cs.cmu.edu/projects/sma/





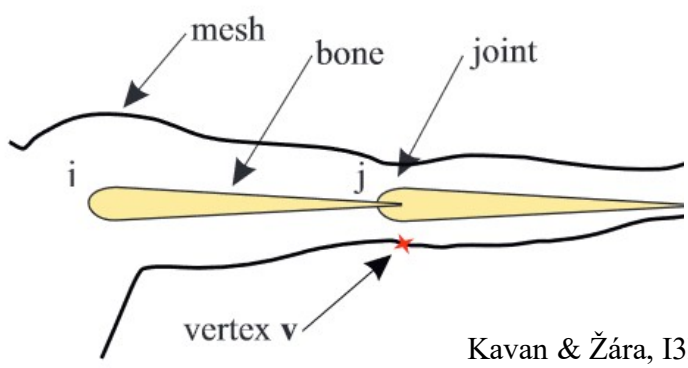
B. H. Le & Z. Deng, SIGGRAPH Course 2014
Skinning: Real-time Shape Deformation

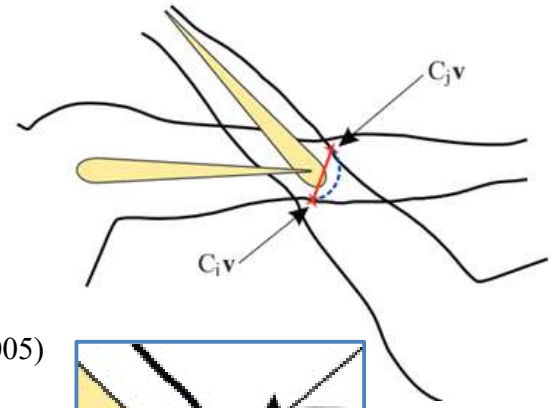
23

Let's now look at skinning – we want to deform mesh-based character models

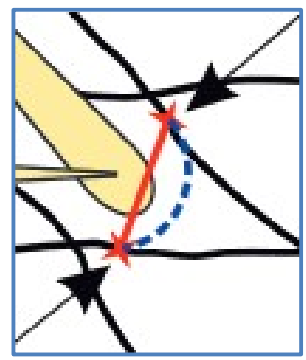
(James and Twigg actually don't use bones … it's an interesting paper …)

The 2014 SigGraph course notes can be found at skinning.org





Kavan & Žára, I3D (2005)



Skinning:
How to interpolate transformations from bones to mesh vertices?

24

We want the mesh vertices to follow the motion of nearby bones.

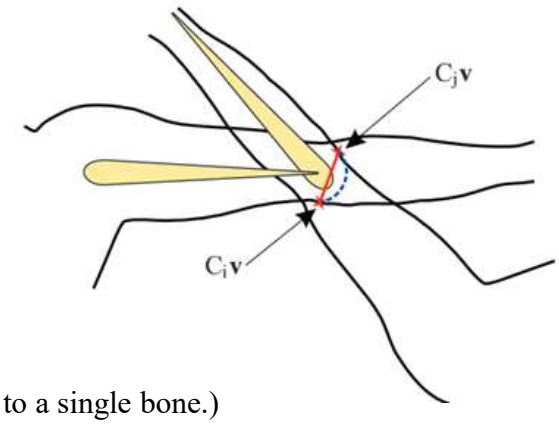
Near joints we want a smooth transition from one bone to the next.

Kavan & Žára, I3D (2005)

Recall: $\mathbf{T}_j^0 = \prod_{i=1}^{j} \mathbf{T}_i^{i-1}$,

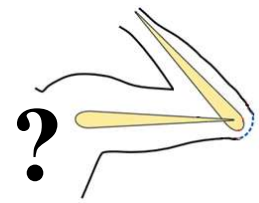
let: $\mathbf{A}_j = (\mathbf{o}_0 \mathbf{x}_0 \mathbf{y}_0 \mathbf{z}_0)\mathbf{T}_j^0$,

and: $\mathbf{F}_j = (\mathbf{o}_0 \mathbf{x}_0 \mathbf{y}_0 \mathbf{z}_0)\tilde{\mathbf{T}}_j^0$.



$\tilde{\mathbf{v}} = \mathbf{F}_j \mathbf{A}_j^{-1} \mathbf{v} = \mathbf{C}_j \mathbf{v}$ (Vertex bound to a single bone.)

$\tilde{\mathbf{v}} = \sum_j w_j \mathbf{C}_j \ \mathbf{v}$ **Linear blend skinning**



25

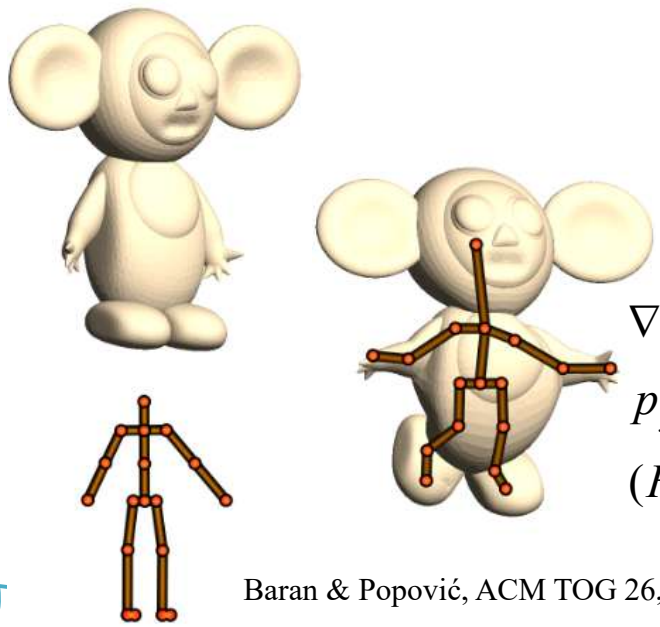
A_j refers to the “rest” or “bind” pose of the character, F_j to the “final” or “deformed” configuration, C_j combines the vertex transformation around bone j.

Linear blend skinning simply computes a weighted sum (weights themselves should sum to 1 – “partition of unity”) of the nearby bone transforms.

This way, we interpolate between bone transforms i and j along the red line … which might not be the best option, but is very easy to implement.

$$\tilde{\mathbf{v}}_i = \sum_j w_{ji} \mathbf{C}_j \ \mathbf{v}_i$$

How to find weights (for every vertex)?



Idea: solve heat diffusion over surface, per bone *j*:

$$\nabla^2 \mathbf{w}_j + \mathbf{H}_j (\mathbf{p}_j - \mathbf{w}_j) = 0$$

$$p_{ji} = 1 \quad \text{if } j \text{ is nearest bone of vertex } i$$

$$(H_j)_{ii} = c / d_{ji}^2$$

Baran & Popović, ACM TOG 26, 3 (2007)

26

Of course assigning skinning weights for all mesh vertices (and all bones) is a bit tedious.

How to find weights automatically? Solve diffusion (surface cotangent Laplacian – we'll come back to that in a future lecture).

Idea: "encourage" weights to be 1 for the nearest bone, and 0 for all others, while maintaining smoothness of weights over the mesh (the diffusion part).

H is a weight on this condition depending on the squared distance to from the vertex to the bone (with basic visibility check).

Summation reveals that these weights have partition of unity – see the paper for details.

$$\tilde{\mathbf{v}}_i = \sum_j w_{ji} \mathbf{C}_j \ \mathbf{v}_i$$

How to find weights (for every vertex)?

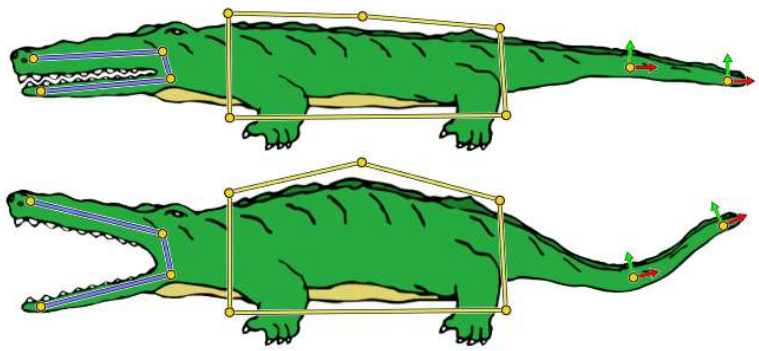
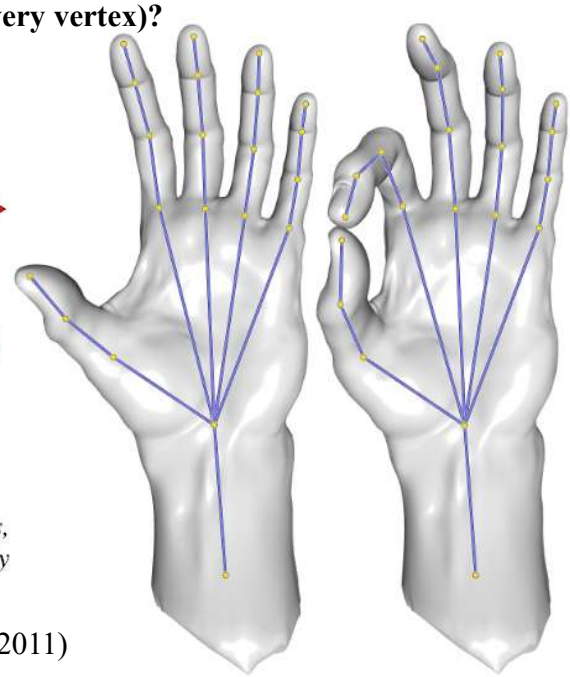


Figure 1: *Bounded biharmonic blending supports points, bones, and cages arranged in an arbitrary configuration. This versatility*

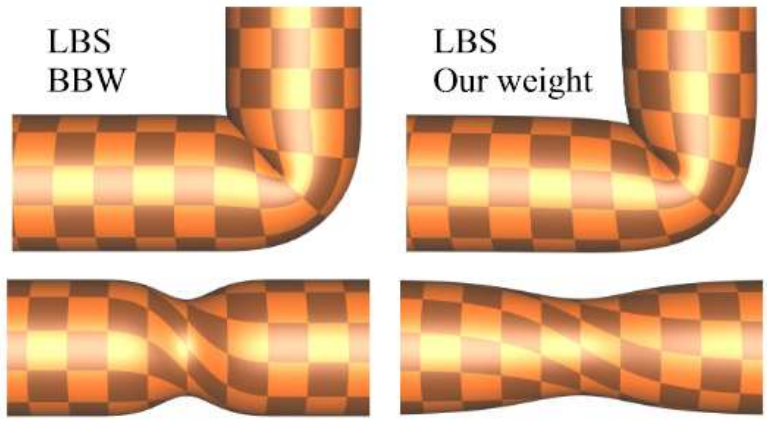


Jacobson et al., ACM TOG 30, 4 (2011)

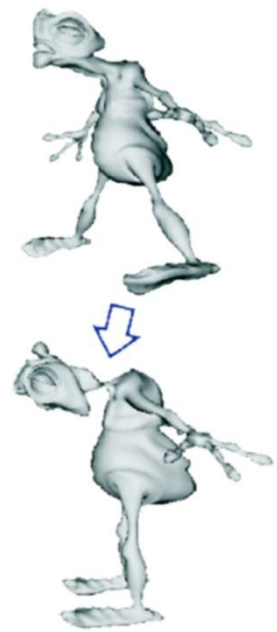
27

Bounded biharmonic weights (BBW) extend this idea to also include cages and handles.

They use a volumetric Laplacian instead, and include bounds for w in [0, 1] and partition-of-unity constraints explicitly -> QP with linear constraints to solve (there are libraries with these solvers available).



Kavan & Sorkine, ACM TOG 31, 6 (2012)



Kavan et al., Graphite (2006)

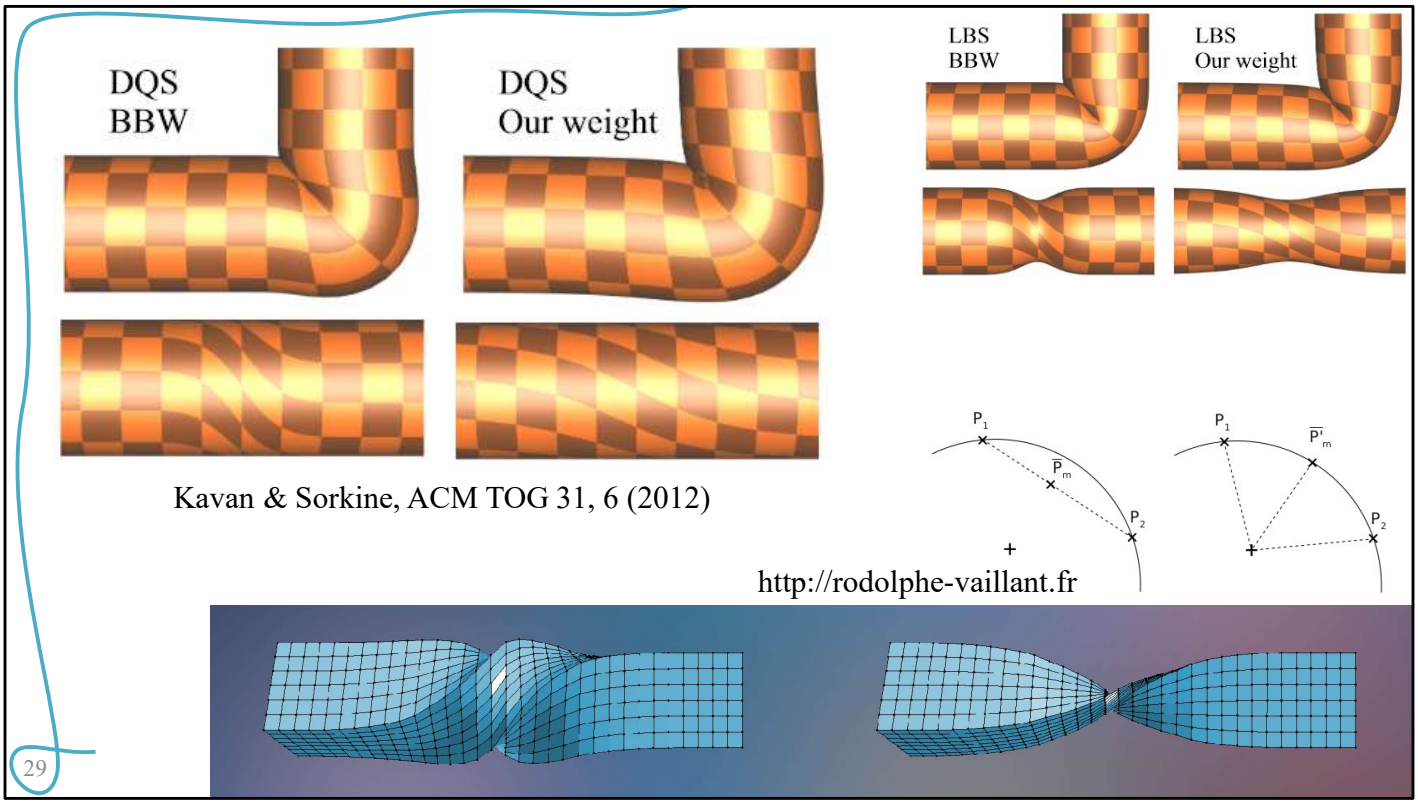
28

Linear blend skinning causes some artefacts, mostly “candy-wrapper” for twists, or pinching at hinges.

These artefacts can, to some extent, be mitigated by more elaborate weight construction.

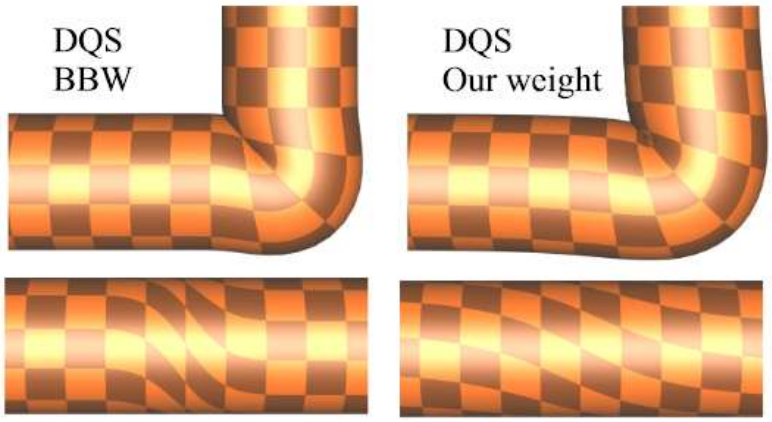
Unfortunately, they’ll never go away completely if we stick to linear blending.

Image: https://igl.ethz.ch/projects/eid/jointdefWeb.pdf



Dual quaternion skinning interpolates rotations more elegantly, thereby fixing candy-wrapper artefacts.

It can, however, introduce some (maybe undesirable) bulging around hinges.

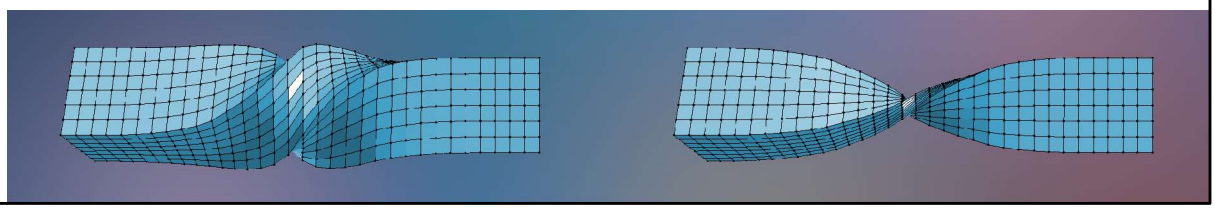


Kavan & Sorkine, ACM TOG 31, 6 (2012)

Rigid transforms with 8 floats
(instead of 16 for matrices).

"Spherical" interpolation
of rotations
(instead of linearized).

Math is complicated,
but code is fast and elegant.



30

Advantages of dual quaternion skinning …

[skipping math here]

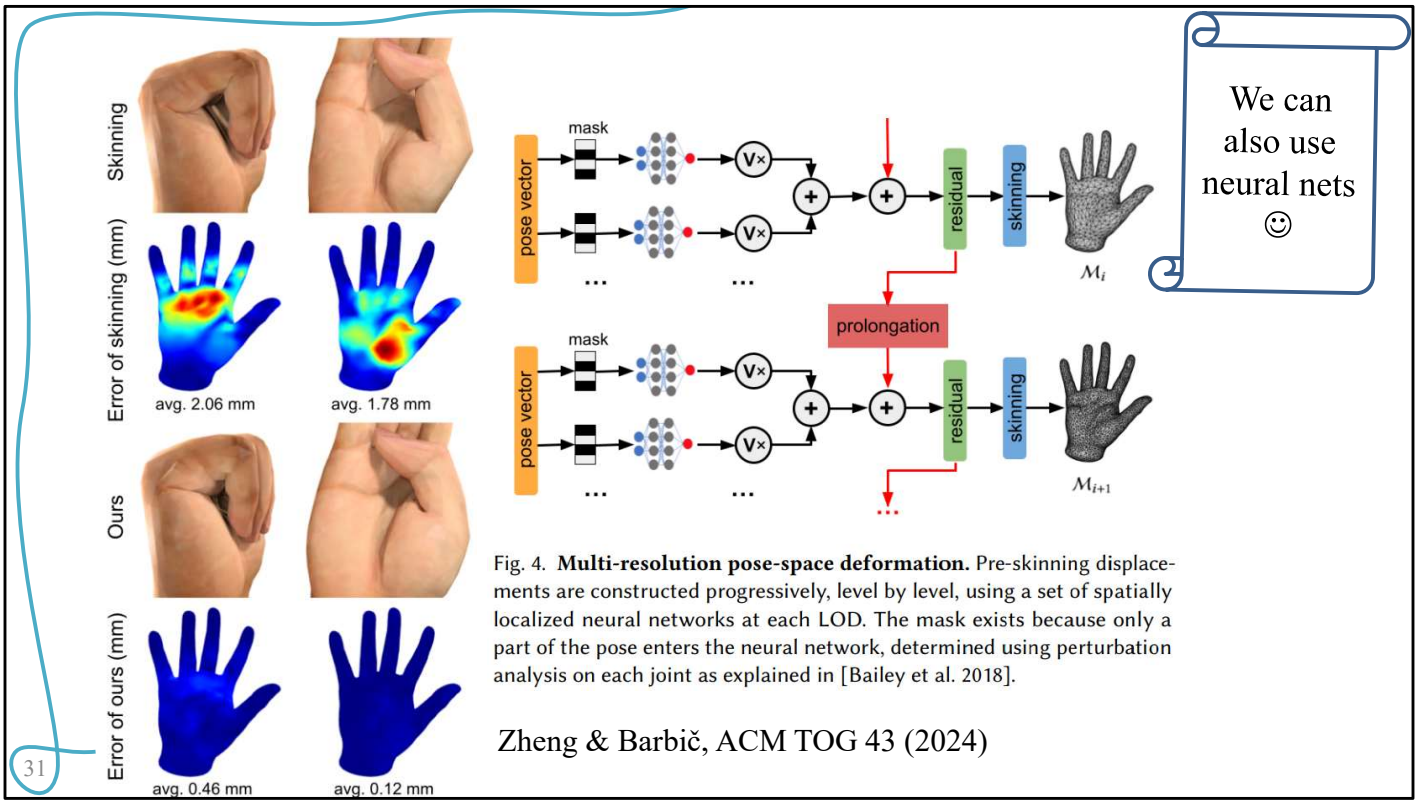


Fig. 4. **Multi-resolution pose-space deformation.** Pre-skinning displacements are constructed progressively, level by level, using a set of spatially localized neural networks at each LOD. The mask exists because only a part of the pose enters the neural network, determined using perturbation analysis on each joint as explained in [Bailey et al. 2018].

Of course there has been work on using neural networks for skinning in recent years.

It takes a bit more time to train using simulated "ground-truth" deformation data, but can then quickly approximate realistic deformations beyond the basic LBS or DQS interpolation spaces.

Check out the paper by Zheng and Barbic here:
https://dl.acm.org/doi/10.1145/3687985

Image: mcvijay.wordpress.com

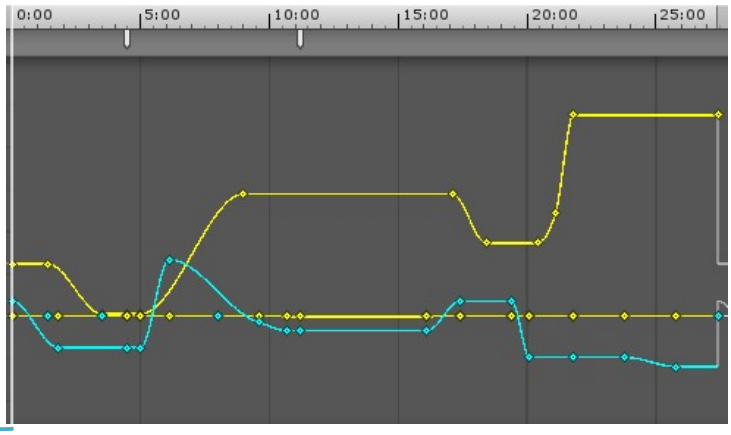


Image: docs.unity3d.com

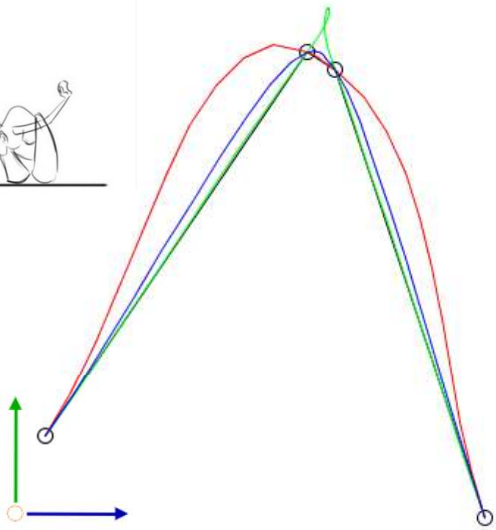
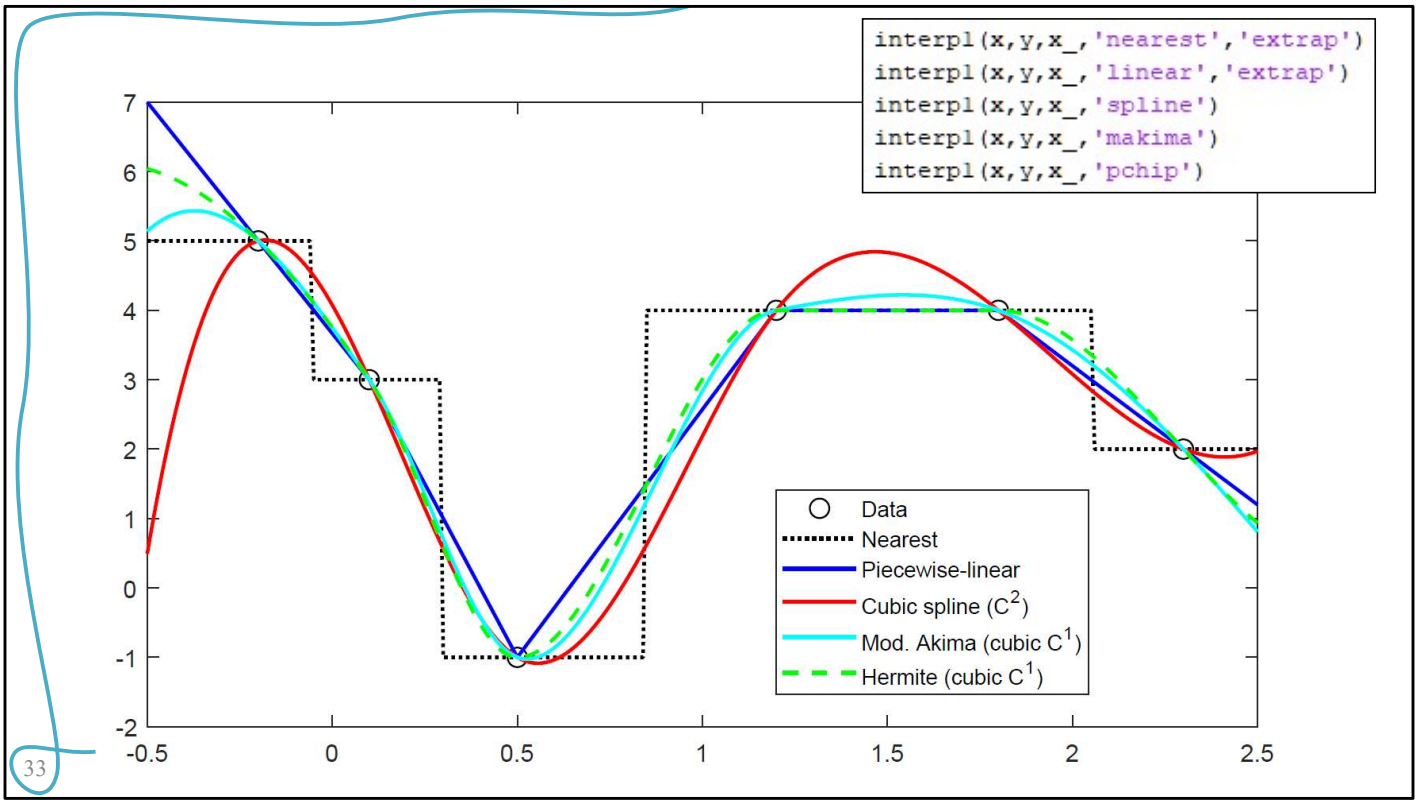


Image: Wikipedia User:Hadunsford

Animation via keyframes and motion curves (linear vs. smooth interpolation, splines)

Right: Catmull-Rom splines with different time parameterization.



Interpolation comes in many shapes … piecewise cubic variants tend to be most popular (smoothness, controllable tangents, relatively easy to implement)

Cubic Hermite Spline

$$y(\mathrm{x}) = h_{00}(x)y_0 + h_{01}(x)y_1 + h_{10}(x)y_0' + h_{11}(x)y_1'$$

Linear combination of cubic basis functions

Coefficients are values and tangents at interval endpoints

Choice of tangents, e.g. Catmull-Rom:

$$y_k' = \frac{1}{2}\frac{y_{k+1} - y_{k-1}}{x_{k+1} - x_{k-1}}$$

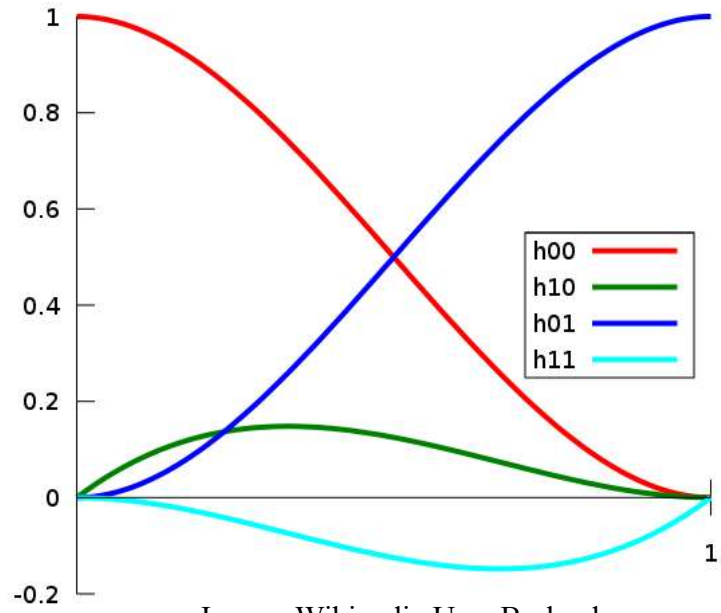


Image: Wikipedia User:Berland

34

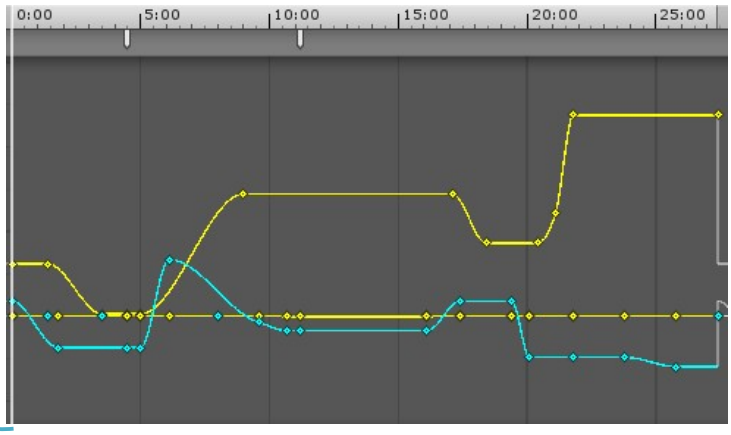
Cubic Hermite splines represent each segment as a combination of four cubic basis functions (image).

Two of them control the function value at each end of the interval respectively, the other two, similarly, control the function gradient at each endpoint.

The gradients (tangents) can be provided by user input, or estimated by various assumptions, notably the Catmull-Rom spline computes them based on the neighbouring data points.

Motion Capture

Drawing motion curves by hand is annoying …



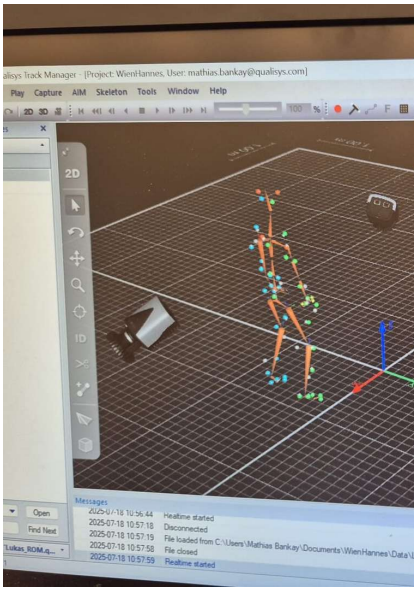
35

Image: docs.unity3d.com

Animation via motion capture … a very brief overview.

General idea: map mocap data to actor skeleton (inverse kinematics) -> map to character skeleton (re-targeting) -> motion of character skeleton (forward kinematics) -> skinning

Motion Capture



Images: Lukas Pezzei

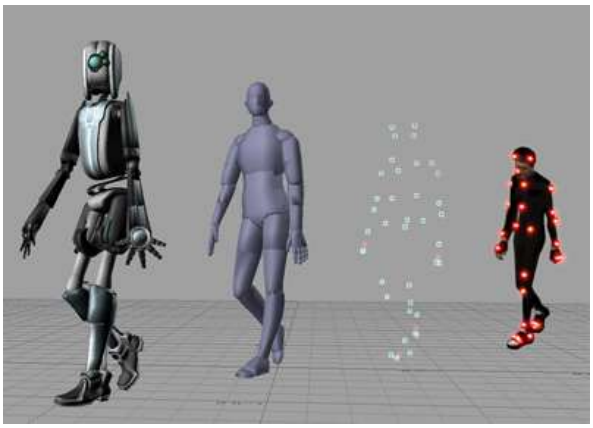


Image: Wikipedia User:Hipocrite

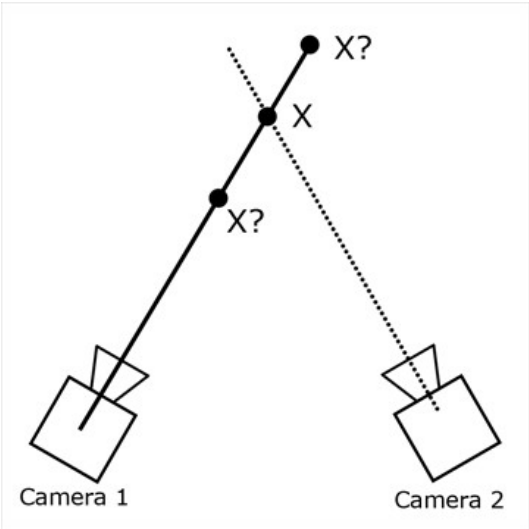


Image: http://physbam.stanford.edu/cs448x

Motion Capture

Basic idea:

Each camera image is a projection of the 3D scene onto a 2D plane.

Inverse projection: find (best matching) marker positions given a set of camera images.

Inverse kinematics: match skeleton to markers.

Image: optitrack.com

37

In practice:

- buy a mocap package with software license, cameras, markers, etc. (fortunately, we already have some downstairs),
- mount cameras (way more than 2, better >10)

…

Image: Lukas Pezzei

Motion Capture

Images: optitrack.com

38

- calibrate (ie. run around with calibration wand, set coordinate frame with triangle plate, then let the software figure out the camera placement)
- record motion
- post process (lots of data clean-up, occluded markers, numerical errors, etc.)

Image: cinemontage.org Joseph Herman

Kholgade et al., SCA 2011

39

Left: motion captured from the actor is transferred to a skeleton suitable for the actor, then re-targeted to the digital character.

Right: some more elaborate motion re-targeting examples.

Thank you!

Questions?